

The Consent Gate

Opening a store now requires the current seller terms, refused server-side and recorded against the exact version. The consent machinery has existed since Phase 0 with nothing calling it — this wires it to a real action, and found that it failed open.

Branch **claude/create-app-repository-1cxsih**

Head **88c4f1d**

1 commit • 15 files

Tests **214 passed • 10 browser**

Mutations caught **6 of 6**

CI run **#13 • 3/3 green**

1

Which action, and why it had to be asked

The brief said to identify the protected action from the blueprint, and to stop rather than choose silently if it was ambiguous. It was ambiguous, so I stopped.

THE BLUEPRINT FIXES	THE BLUEPRINT DOES NOT FIX
The principle — "users accept Afrinext's terms before using the commercial functionality those terms govern" The mechanism — "consent is a gate, not a banner: an action whose terms the user has not accepted at the current effective version is refused server-side" The line — consent establishes layer B, never layer G	Which action requires which document. Seven kinds are seeded; the <code>method</code> column enumerates <code>signup</code>, <code>seller_onboarding</code>, <code>reaccept</code>, <code>checkout</code> — which implies gates exist without saying where they attach.

One candidate mattered more than the rest: gating **signup** would have meant modifying the Phase 1 OTP flow, which the same brief twice forbids without approval. Choosing it silently would have been a change to authentication smuggled in as a consent feature.

DECIDED, AND IMPLEMENTED EXACTLY AS DECIDED

The protected action is `createStore`. The document is `seller_terms`, alone — the one whose subject matter is the action being taken. The general terms are a signup gate, which is a different flow and a later milestone.

Publishing a store and adding products are deliberately *not* gated again: you cannot publish a store you were never allowed to open, so a second check would add an enforcement point to maintain without adding a guarantee. That choice is asserted by a test, not left implicit.

What already existed, and what this adds

Phase 0 built the consent machinery and Phase 1 left it untouched. Before this milestone it had **zero call sites outside its own test file**. The instruction not to invent a parallel system was therefore easy to honour — there was a complete one to wire up.

ALREADY THERE	ADDED
<code>currentVersions</code> — effective-dated, country-specific beats global <code>outstandingConsents</code> <code>requireConsent</code> — the gate <code>recordConsent</code> — append-only <code>consentHistory</code> three tables, and <code>consent.required</code> → HTTP 451	<code>requireConsent</code> made to fail closed <code>ConsentUnavailableError</code> → HTTP 503 <code>acceptCurrentVersions</code> — server-resolved acceptance <code>requireSellerConsent</code> in the catalogue the gate call inside <code>createStore</code> consent panel, Server Action, <code>/api/v1/consent</code>

FOUND WHILE WIRING IT, NOT BY A TEST THAT ALREADY EXISTED

`requireConsent` asked `outstandingConsents` what was unaccepted and threw if the list was non-empty. But a document with **no effective version in the user's locale** produces an empty list — indistinguishable, to that gate, from "everything has been accepted".

So a missing translation, a document nobody published yet, or a locale with no catalogue would have switched consent off *silently*. Nothing would have logged. The action would simply have proceeded.

It now fails closed. Every requested kind must resolve to a version in force, or the action is refused with a distinct error:

```
// `outstandingConsents` can only report what it found, so a document with no
// effective version in the requested locale produces an empty list —
// indistinguishable, to a naive gate, from "everything is accepted".
const resolved = new Set(current.map((c) => c.kind));
const missing = kinds.filter((k) => !resolved.has(k));
if (missing.length > 0) throw new ConsentUnavailableError(missing, options.locale);
```

The two refusals are deliberately different errors, because the remedies are different. `consent.required` is a **451** and the person can act on it. `consent.unavailable` is a **503** — the person can do nothing, and an operator must publish the document. Returning 4xx there would blame the caller for our gap.

Two tests cover it: a withdrawn translation, and a version whose effective date is in the future.

Two things a transport may not decide

DECISION	WHERE IT IS TAKEN	WHAT IT PREVENTS
Which locale's document applies	Read from the actor's own <code>users</code> row inside the core layer	A caller naming a locale with no published document. Before the fail-closed fix that alone opened the gate; even now, which terms bind a person is not a client's decision.
Which version is being accepted	<code>acceptCurrentVersions</code> takes kinds , never a version id, and resolves what is in force itself	A client recording acceptance of a superseded version — real-looking, satisfying nothing, and leaving the gate closed for a reason nobody can see.

Both are asserted directly. One test records an old version id by hand — exactly what a client that chose its own version could do — and confirms the gate stays shut.

The panel is presentation, and the tests say so

You were explicit that displaying a checkbox is not enforcement. The seller page renders a consent panel naming the document, its version and its locale — and that panel is **irrelevant to the outcome**.

BROWSER TEST	WHAT IT PROVES
Posts to <code>/api/v1/stores</code> from a session that never loaded the panel	451, code <code>consent.required</code> , message naming <code>seller_terms</code> , and zero rows written.
Deletes the panel from the DOM , as anyone with dev tools could, then posts	451 again. The server has not changed its mind.
Accepts through the UI	Exactly one record: <code>seller_terms</code> <code>0.0.0-placeholder</code> <code>seller_onboarding</code> , with the user agent captured. The same call that was 451 now returns 201.
Accepts through the API , then uses the form path	The form path is open without ever having accepted anything itself — one gate, two transports. Accepting twice records once.
An operator publishes new terms mid-session	The next action is 451 again, the screen re-prompts without anyone being told to, and the earlier acceptance is still on record.

A refusal that does not say what to present forces the client to guess, so the error carries the document kinds. That is the only thing the server tells the interface.

Mutation evidence — six, all caught

DELIBERATE DEFECT	FAILING TESTS
<code>createStore</code> stops calling the gate	9
The fail-open is restored (missing document = pass)	2
Any acceptance counts; the version is ignored	3
The locale is taken from the caller, not the user row	30
Acceptance records a stale version instead of the current one	29
The server gate is removed while the UI still hides the form	4 OF 5 BROWSER TESTS

TWO HONEST NOTES ON THIS TABLE

The first run was invalid. The mutations were run before I noticed that milestone 2's catalogue tests had gone red at baseline — the gate genuinely changed what `createStore` requires, and their helper had not accepted anything. A mutation run against a red baseline proves nothing. The helper now carries the acceptance, the baseline is 48 green, and every mutation above was re-run against it.

Mutation 6 leaves one browser test passing: the one asserting that the API and the form agree. With the gate gone they agree that everything is allowed. It is testing consistency between transports, not the gate, and it should not be credited with catching its removal.

A harness gap the fail-closed test exposed

Testing that the gate refuses when a document is unavailable means taking the document away.

`resetData` restored versions a test had *added* but not ones it had *deleted*, so that single test left every later consent check running against a database where `seller_terms` no longer existed — failing for entirely the wrong reason.

The fix is in the harness, not in the one test that noticed: the baseline is now restored in both directions, including re-creating deleted placeholders and undoing any shift of their effective date.

The test itself also changed, and the reason is worth recording: consent records are append-only, so an accepted version **cannot be deleted** — the foreign key holds it in place. Withdrawing a document in reality means making it no longer effective, which is what an operator would actually do, so that is what the test does.

GitHub CI run #13 on `88c4f1d`: `LINT · TYPECHECK · TEST` `BROWSER` `DEPENDENCY SCAN` — all three jobs green, on a runner that rebuilds the schema from zero.

Locally, on the same commit

```
pnpm install --frozen-lockfile    exit 0
db reset → migrate → seed        from zero · 8 roles, 7 legal documents
pnpm lint                        exit 0
pnpm typecheck                   exit 0
pnpm test                        214 passed | 3 skipped (18 files)    was 196 / 17
pnpm build                       exit 0
playwright test                  10 passed                                was 5
ledger concurrency               6 passed
reconcile-check                  clean, zero drift
pnpm audit --audit-level high    No known vulnerabilities found
```

New tests: **18 domain** in `consent/gate.test.ts` and **5 browser** in `consent.spec.ts`, numbered against the nine requirements in the brief.

Files changed — 15, all consent

FILE	WHY
<code>core/consent/gate.test.ts</code>	New. 18 tests, written adversarially.
<code>core/consent/index.ts</code>	Fail-closed gate, <code>ConsentUnavailableError</code> , <code>acceptCurrentVersions</code> .
<code>core/catalog/index.ts</code>	<code>SELLER_CONSENT_KINDS</code> , <code>requireSellerConsent</code> , and the gate call in <code>createStore</code> .
<code>web/e2e/consent.spec.ts</code>	New. The five browser assertions.
<code>web/lib/consent.ts</code>	New. Server-resolved legal context and acceptance evidence.
<code>web/lib/consent-actions.ts</code>	New. The Server Action.
<code>web/api/v1/consent/route.ts</code>	New. GET outstanding, POST accept.
<code>web/components/ConsentGate.tsx</code>	New. The panel.
<code>web/[locale]/sell/page.tsx</code>	Renders the panel when something is outstanding.
<code>web/lib/api.ts</code>	<code>consent.unavailable</code> → 503.
<code>core/test/harness.ts</code>	The baseline restoration described in section 7.
<code>core/catalog/catalog.test.ts</code>	The helper now accepts the terms.
<code>web/e2e/catalog.spec.ts</code>	The journey includes accepting.
<code>i18n fr.json · en.json</code>	5 keys per locale.

No migration and no new dependency. The consent schema already existed; `pnpm-lock.yaml` is untouched. Nothing from the OTP model, the ledger, the money model, scoped RBAC, the browser-CI foundation or the public visibility rules appears in the diff.

ITEM	STATE
The documents have no text. Every version is 0.0.0-placeholder and the panel says so rather than rendering an empty box that looks like terms. The gate works with any text; writing it is a legal deliverable.	BLOCKS REAL USE, NOT THE MECHANISM
Signup is not gated. terms_of_use and privacy_policy are accepted by nobody, because the only signup path is the OTP flow and touching it needs your approval.	NEEDS A DECISION
Checkout, payout and withdrawal gates are unbuilt — payment_refund_policy, payout_terms and referral_terms govern actions that do not exist yet.	BY INSTRUCTION
Withdrawing consent is not implemented. Records are append-only and there is no revocation path.	A REAL GAP UNDER DATA-PROTECTION LAW
No admin screen publishes a new document version — it is an SQL insert.	ACCEPTED FOR NOW
Gated seller model: nothing grants seller automatically, as you decided. Development and testing assign it directly.	AS INSTRUCTED

The second row is the one I would settle next. Nobody has accepted the terms of use or the privacy policy — including buyers, who will never open a store and so will never meet the only gate that exists. That is a real gap and closing it means touching the sign-in flow.

Proposed next: milestone 4 — checkout against the mock provider, or the signup consent gate first if you would rather close that gap before more actions exist. **Not started.**

Phase 2 · milestone 3 of 5 · branch `claude/create-app-repository-1cxsih` · head `88c4f1d`